

前缀和与差分算法完全指南

算法学习笔记

2026 年 5 月 13 日

目录

1	前缀和算法基础	2
1.1	什么是前缀和？	2
1.2	一维前缀和模板	2
1.3	前缀和的核心优势	4
1.4	前缀和的变体	4
2	二维前缀和	5
2.1	二维前缀和原理	5
2.2	二维前缀和模板	6
3	差分算法基础	8
3.1	什么是差分？	8
3.2	一维差分模板	9
3.3	差分与前缀和的关系	11
4	二维差分	11
4.1	二维差分原理	11
5	经典例题详解	13
5.1	例题 1：和为 K 的子数组	13
5.2	例题 2：区域和检索（二维前缀和）	15
5.3	例题 3：航班预订统计	16
5.4	例题 4：拼车问题	18
6	进阶题目详解	19
6.1	进阶题 1：乘积最大子数组	19
6.2	进阶题 2：连续子数组的最大和	21

6.3	进阶题 3: 矩形区域不超过 K 的最大和	22
6.4	进阶题 4: 区间加法 (二维差分应用)	24
6.5	进阶题 5: 按权重随机选择	26
7	高级技巧与优化	27
7.1	技巧 1: 前缀和 + 单调栈	27
7.2	技巧 2: 前缀和 + 同余定理	29
7.3	技巧 3: 前缀统计 (计数排序思想)	30
8	总结与对比	31
9	练习题推荐	32

1 前缀和算法基础

1.1 什么是前缀和？

前缀和 (Prefix Sum) 是一种预处理技术, 通过预先计算数组的前缀和, 可以在 $O(1)$ 时间内回答任意区间和的查询。

前缀和的核心思想

一句话概括: 提前计算好前 i 个元素的和, 使得区间求和从 $O(n)$ 降到 $O(1)$ 。

数学定义: 给定数组 $a[0], a[1], \dots, a[n-1]$, 定义前缀和数组 $pre[i]$:

$$pre[i] = \sum_{k=0}^i a[k] = a[0] + a[1] + \dots + a[i]$$

特别地, $pre[-1] = 0$ (或设置 $pre[0] = 0$ 作为哨兵)

区间求和公式:

$$\sum_{k=l}^r a[k] = pre[r] - pre[l-1]$$

1.2 一维前缀和模板

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      // 构建前缀和数组
6      vector<int> buildPrefixSum(vector<int>& nums) {
7          int n = nums.size();
8          vector<int> pre(n + 1, 0); // 多开一个空间, pre[0]=0
9
10         for (int i = 0; i < n; i++) {
11             pre[i + 1] = pre[i] + nums[i];
12         }
13
14         return pre;
15     }
16
17     // 查询区间 [l, r] 的和 (0-indexed)
18     int rangeSum(vector<int>& pre, int l, int r) {
19         return pre[r + 1] - pre[l];
```

```

20     }
21
22     int main() {
23         vector<int> nums = {3, 5, 2, -1, 6, 8, -3, 4};
24
25         // 构建前缀和
26         vector<int> pre = buildPrefixSum(nums);
27
28         cout << "原始数组: ";
29         for (int x : nums) cout << x << " ";
30         cout << endl;
31
32         cout << "前缀和数组: ";
33         for (int x : pre) cout << x << " ";
34         cout << endl;
35
36         // 查询区间和
37         cout << "区间 [0, 2] 的和: " << rangeSum(pre, 0, 2) << endl
38             ; // 3+5+2=10
39         cout << "区间 [1, 4] 的和: " << rangeSum(pre, 1, 4) << endl
40             ; // 5+2-1+6=12
41         cout << "区间 [3, 7] 的和: " << rangeSum(pre, 3, 7) << endl
42             ; // -1+6+8-3+4=14
43
44         return 0;
45     }
46
47     /* 输出:
48     原始数组: 3 5 2 -1 6 8 -3 4
49     前缀和数组: 0 3 8 10 9 15 23 20 24
50     区间 [0, 2] 的和: 10
51     区间 [1, 4] 的和: 12
52     区间 [3, 7] 的和: 14
53     */

```

Listing 1: 一维前缀和 - 基础模板

1.3 前缀和的核心优势

为什么要用前缀和？

操作	不使用前缀和	使用前缀和
单次区间查询	$O(n)$ 遍历求和	$O(1)$ 公式计算
m 次区间查询	$O(m \times n)$ 可能超时	$O(n + m)$ 高效
空间复杂度	$O(1)$	$O(n)$ 额外空间

典型场景：当需要多次查询区间和时，前缀和可以大幅降低时间复杂度。

1.4 前缀和的变体

```
1      #include <iostream>
2      #include <vector>
3      #include <unordered_map>
4      using namespace std;
5
6      // 变体1：前缀异或和
7      vector<int> buildPrefixXor(vector<int>& nums) {
8          int n = nums.size();
9          vector<int> preXor(n + 1, 0);
10         for (int i = 0; i < n; i++) {
11             preXor[i + 1] = preXor[i] ^ nums[i];
12         }
13         return preXor;
14     }
15
16     // 查询区间异或值
17     int rangeXor(vector<int>& preXor, int l, int r) {
18         return preXor[r + 1] ^ preXor[l];
19     }
20
21     // 变体2：前缀积（注意溢出和除零问题）
22     vector<long long> buildPrefixProduct(vector<int>& nums) {
23         int n = nums.size();
24         vector<long long> preProd(n + 1, 1); // 乘积初始化为1
25         for (int i = 0; i < n; i++) {
26             preProd[i + 1] = preProd[i] * nums[i];
27         }
28     }
```

```

28         return preProd;
29     }
30
31     // 变体3: 前缀最值 (前缀最大值)
32     vector<int> buildPrefixMax(vector<int>& nums) {
33         int n = nums.size();
34         vector<int> preMax(n + 1, INT_MIN);
35         for (int i = 0; i < n; i++) {
36             preMax[i + 1] = max(preMax[i], nums[i]);
37         }
38         return preMax;
39     }
40
41     int main() {
42         vector<int> nums = {3, 5, 2, 7, 1, 8, 4};
43
44         // 前缀异或和
45         vector<int> preXor = buildPrefixXor(nums);
46         cout << "区间 [1, 3] 的异或值: " << rangeXor(preXor, 1, 3)
47             << endl;
48
49         // 前缀最大值
50         vector<int> preMax = buildPrefixMax(nums);
51         cout << "前5个元素的最大值: " << preMax[5] << endl;
52
53         return 0;
54     }

```

Listing 2: 前缀和变体 - 多种应用

2 二维前缀和

2.1 二维前缀和原理

二维前缀和将前缀和扩展到了二维矩阵，可以在 $O(1)$ 时间内计算任意子矩阵的和。

二维前缀和公式

构建公式：

$$pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i-1][j-1]$$

子矩阵查询公式：左上角 $(x1, y1)$ ，右下角 $(x2, y2)$ 的子矩阵和：

$$sum = pre[x2][y2] - pre[x1-1][y2] - pre[x2][y1-1] + pre[x1-1][y1-1]$$

图解：

$pre[x2][y2]$	$- pre[x1-1][y2]$	$- pre[x2][y1-1]$	$+ pre[x1-1][y1-1]$
	$=$	$-$	$+$
全部区域		$-$ 上方区域	$-$ 左方区域
			$+$ 左上角(重叠)

2.2 二维前缀和模板

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      class NumMatrix {
6      private:
7          vector<vector<int>> pre;    // 二维前缀和数组
8
9      public:
10         // 构建二维前缀和
11         NumMatrix(vector<vector<int>>& matrix) {
12             if (matrix.empty()) return;
13
14             int n = matrix.size();
15             int m = matrix[0].size();
16
17             // 多开一行一列，方便处理边界
18             pre.resize(n + 1, vector<int>(m + 1, 0));
```

```

19
20     for (int i = 1; i <= n; i++) {
21         for (int j = 1; j <= m; j++) {
22             pre[i][j] = pre[i-1][j] + pre[i][j-1]
23             - pre[i-1][j-1] + matrix[i-1][j-1];
24         }
25     }
26 }
27
28 // 查询子矩阵和: (row1, col1) 到 (row2, col2) (0-indexed)
29 int sumRegion(int row1, int col1, int row2, int col2) {
30     // 转换为1-indexed
31     row1++; col1++; row2++; col2++;
32
33     return pre[row2][col2]
34     - pre[row1-1][col2]
35     - pre[row2][col1-1]
36     + pre[row1-1][col1-1];
37 }
38
39 // 打印前缀和矩阵 (调试用)
40 void print() {
41     for (int i = 0; i < pre.size(); i++) {
42         for (int j = 0; j < pre[0].size(); j++) {
43             cout << pre[i][j] << "\t";
44         }
45         cout << endl;
46     }
47 }
48 };
49
50 int main() {
51     vector<vector<int>> matrix = {
52         {3, 0, 1, 4, 2},
53         {5, 6, 3, 2, 1},
54         {1, 2, 0, 1, 5},
55         {4, 1, 0, 1, 7},
56         {1, 0, 3, 0, 5}
57     };
58

```



```

59     NumMatrix nm(matrix);
60
61     cout << "原始矩阵:" << endl;
62     for (auto& row : matrix) {
63         for (int x : row) cout << x << " ";
64         cout << endl;
65     }
66
67     cout << "\n前缀和矩阵:" << endl;
68     nm.print();
69
70     // 查询子矩阵和
71     cout << "\n子矩阵(2,1)到(4,3)的和: "
72     << nm.sumRegion(2, 1, 4, 3) << endl; // 2+0+1 + 1+0+1 +
73     0+3+0 = 8
74
75     cout << "子矩阵(1,1)到(2,2)的和: "
76     << nm.sumRegion(1, 1, 2, 2) << endl; // 6+3+2+0 = 11
77
78     return 0;

```

Listing 3: 二维前缀和 - 完整模板

3 差分算法基础

3.1 什么是差分？

差分（Difference Array）是前缀和的逆运算，用于快速对区间进行增减操作。

差分的核心思想

一句话概括：将对区间的修改操作转化为对两个端点的修改，使得区间修改从 $O(n)$ 降到 $O(1)$ 。

数学定义：给定数组 $a[0], a[1], \dots, a[n-1]$ ，定义差分数组 $diff[i]$ ：

$$diff[0] = a[0]$$

$$diff[i] = a[i] - a[i-1] \quad (i \geq 1)$$

区间修改操作：要对区间 $[l, r]$ 的所有元素加上 val ：

$$diff[l] += val$$

$$diff[r+1] -= val$$

还原原数组：对差分数组求前缀和即可还原：

$$a[i] = \sum_{k=0}^i diff[k]$$

3.2 一维差分模板

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      class Difference {
6      private:
7          vector<int> diff; // 差分数组
8
9      public:
10         // 初始化差分数组
11         Difference(vector<int>& nums) {
12             int n = nums.size();
13             diff.resize(n);
14
15             if (n > 0) {
16                 diff[0] = nums[0];
17                 for (int i = 1; i < n; i++) {
18                     diff[i] = nums[i] - nums[i-1];
19                 }
20             }
21         }
22     };
```

```

20         }
21     }
22
23     // 对区间 [l, r] 的所有元素增加 val
24     void increment(int l, int r, int val) {
25         diff[l] += val;
26         if (r + 1 < diff.size()) {
27             diff[r + 1] -= val;
28         }
29     }
30
31     // 还原原数组
32     vector<int> result() {
33         vector<int> res(diff.size());
34         res[0] = diff[0];
35         for (int i = 1; i < diff.size(); i++) {
36             res[i] = res[i-1] + diff[i];
37         }
38         return res;
39     }
40 };
41
42 int main() {
43     vector<int> nums = {1, 2, 3, 4, 5, 6, 7, 8};
44
45     cout << "原始数组: ";
46     for (int x : nums) cout << x << " ";
47     cout << endl;
48
49     Difference d(nums);
50
51     // 执行区间修改
52     d.increment(1, 3, 2);    // 下标1到3的元素增加2
53     d.increment(2, 5, -3);  // 下标2到5的元素减少3
54     d.increment(0, 7, 1);   // 所有元素增加1
55
56     // 获取最终结果
57     vector<int> result = d.result();
58
59     cout << "修改后的数组: ";

```

```

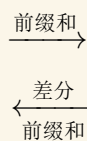
60     for (int x : result) cout << x << " ";
61     cout << endl;
62
63     // 手动验证:
64     // 原数组: [1, 2, 3, 4, 5, 6, 7, 8]
65     // 操作1(1,3,+2): [1, 4, 5, 6, 5, 6, 7, 8]
66     // 操作2(2,5,-3): [1, 4, 2, 3, 2, 3, 7, 8]
67     // 操作3(0,7,+1): [2, 5, 3, 4, 3, 4, 8, 9]
68
69     return 0;
70 }

```

Listing 4: 一维差分 - 完整模板

3.3 差分与前缀和的关系

前缀和与差分的对偶关系



- 前缀和: 快速查询区间和 (静态查询)
- 差分: 快速修改区间值 (动态修改)
- 前缀和 + 差分: 同时支持区间修改和区间查询 (树状数组的核心思想)

4 二维差分

4.1 二维差分原理

二维差分用于在 $O(1)$ 时间内对二维矩阵的子矩阵进行增减操作。

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     // 二维差分
6     void rangeAdd(vector<vector<int>>& diff,
7     int x1, int y1, int x2, int y2, int val) {

```

```

8         diff[x1][y1] += val;
9         diff[x2+1][y1] -= val;
10        diff[x1][y2+1] -= val;
11        diff[x2+1][y2+1] += val;
12    }
13
14    // 还原原矩阵（对差分矩阵求二维前缀和）
15    vector<vector<int>> getOriginal(vector<vector<int>>& diff) {
16        int n = diff.size();
17        int m = diff[0].size();
18
19        vector<vector<int>> res(n, vector<int>(m, 0));
20
21        for (int i = 0; i < n; i++) {
22            for (int j = 0; j < m; j++) {
23                res[i][j] = diff[i][j];
24                if (i > 0) res[i][j] += res[i-1][j];
25                if (j > 0) res[i][j] += res[i][j-1];
26                if (i > 0 && j > 0) res[i][j] -= res[i-1][j-1];
27            }
28        }
29
30        return res;
31    }
32
33    int main() {
34        int n = 4, m = 5;
35        // diff多开一行一列处理边界
36        vector<vector<int>> diff(n + 1, vector<int>(m + 1, 0));
37
38        // 执行几个子矩阵修改操作
39        rangeAdd(diff, 0, 0, 2, 2, 3); // 左上角3x3区域加3
40        rangeAdd(diff, 1, 1, 3, 3, -2); // 中间区域减2
41
42        // 还原矩阵
43        auto result = getOriginal(diff);
44
45        cout << "修改后的矩阵:" << endl;
46        for (int i = 0; i < n; i++) {
47            for (int j = 0; j < m; j++) {

```

```

48         cout << result[i][j] << " ";
49     }
50     cout << endl;
51 }
52
53 return 0;
54 }

```

Listing 5: 二维差分 - 完整模板

5 经典例题详解

5.1 例题 1：和为 K 的子数组

题目描述：给定一个整数数组和一个整数 k ，求该数组中和为 k 的连续子数组的个数。

解题思路：

- 使用前缀和 + 哈希表
- 对于每个位置 i ，计算当前前缀和 $pre[i]$
- 如果之前有前缀和为 $pre[i] - k$ 的位置 j ，则 $[j + 1, i]$ 的和为 k
- 用哈希表记录每个前缀和出现的次数

```

1    #include <iostream>
2    #include <vector>
3    #include <unordered_map>
4    using namespace std;
5
6    int subarraySum(vector<int>& nums, int k) {
7        // 哈希表：前缀和 -> 出现次数
8        unordered_map<int, int> preSumCount;
9
10       // 初始化：前缀和为0出现1次（处理从开头开始的子数组）
11       preSumCount[0] = 1;
12
13       int curSum = 0; // 当前前缀和
14       int count = 0; // 结果
15

```

```

16     for (int num : nums) {
17         curSum += num; // 计算当前前缀和
18
19         // 如果之前存在前缀和 = curSum - k
20         // 那么从那个位置到当前位置的子数组和为k
21         if (preSumCount.find(curSum - k) != preSumCount.end())
22             {
23                 count += preSumCount[curSum - k];
24             }
25
26         // 更新当前前缀和的计数
27         preSumCount[curSum]++;
28     }
29
30     return count;
31 }
32
33 int main() {
34     vector<int> nums = {1, 1, 1};
35     int k = 2;
36
37     cout << "数组: ";
38     for (int x : nums) cout << x << " ";
39     cout << endl;
40     cout << "和为" << k << "的子数组个数: "
41     << subarraySum(nums, k) << endl;
42     // 输出: 2 (子数组 [1,1] 出现两次)
43
44     nums = {1, 2, 3};
45     k = 3;
46     cout << "\n数组: ";
47     for (int x : nums) cout << x << " ";
48     cout << endl;
49     cout << "和为" << k << "的子数组个数: "
50     << subarraySum(nums, k) << endl;
51     // 输出: 2 ([1,2] 和 [3])
52
53     return 0;
54 }

```

5.2 例题 2: 区域和检索 (二维前缀和)

题目描述: 实现一个类, 支持查询二维矩阵中任意子矩阵的元素和。

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      class NumMatrix {
6      private:
7          vector<vector<int>> pre;
8
9      public:
10         NumMatrix(vector<vector<int>>& matrix) {
11             if (matrix.empty()) return;
12
13             int n = matrix.size();
14             int m = matrix[0].size();
15             pre.assign(n + 1, vector<int>(m + 1, 0));
16
17             for (int i = 1; i <= n; i++) {
18                 for (int j = 1; j <= m; j++) {
19                     pre[i][j] = pre[i-1][j] + pre[i][j-1]
20                         - pre[i-1][j-1] + matrix[i-1][j-1];
21                 }
22             }
23         }
24
25         int sumRegion(int row1, int col1, int row2, int col2) {
26             row1++; col1++; row2++; col2++;
27             return pre[row2][col2]
28                 - pre[row1-1][col2]
29                 - pre[row2][col1-1]
30                 + pre[row1-1][col1-1];
31         }
32     };
33
```



```

34     int main() {
35         vector<vector<int>> matrix = {
36             {3, 0, 1, 4, 2},
37             {5, 6, 3, 2, 1},
38             {1, 2, 0, 1, 5},
39             {4, 1, 0, 1, 7},
40             {1, 0, 3, 0, 5}
41         };
42
43         NumMatrix nm(matrix);
44
45         cout << "区域(2,1)到(4,3)的和: "
46         << nm.sumRegion(2, 1, 4, 3) << endl; // 8
47         cout << "区域(1,1)到(2,2)的和: "
48         << nm.sumRegion(1, 1, 2, 2) << endl; // 11
49
50         return 0;
51     }

```

Listing 7: 例题 2: 二维区域和检索

5.3 例题 3: 航班预订统计

题目描述: 有 n 个航班, 预定记录 $bookings[i] = [first_i, last_i, seats_i]$ 表示从航班 $first_i$ 到 $last_i$ 的每个航班预订了 $seats_i$ 个座位。返回每个航班预订的座位总数。

解题思路: 这是典型的差分数组应用! 多次区间修改, 最后查询所有位置的值。

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     vector<int> corpFlightBookings(vector<vector<int>>& bookings,
6         int n) {
7         // 差分数组
8         vector<int> diff(n, 0);
9
10        // 处理每次预订
11        for (auto& booking : bookings) {
12            int first = booking[0] - 1; // 转换为0-indexed
13            int last = booking[1] - 1;
14            int seats = booking[2];

```

```

14
15         // 差分修改
16         diff[first] += seats;
17         if (last + 1 < n) {
18             diff[last + 1] -= seats;
19         }
20     }
21
22     // 还原结果（差分数组求前缀和）
23     vector<int> result(n);
24     result[0] = diff[0];
25     for (int i = 1; i < n; i++) {
26         result[i] = result[i-1] + diff[i];
27     }
28
29     return result;
30 }
31
32 int main() {
33     vector<vector<int>> bookings = {
34         {1, 2, 10},
35         {2, 3, 20},
36         {2, 5, 25}
37     };
38     int n = 5;
39
40     vector<int> result = corpFlightBookings(bookings, n);
41
42     cout << "各航班预订座位数：";
43     for (int seats : result) {
44         cout << seats << " ";
45     }
46     cout << endl;
47     // 输出：10 55 45 25 25
48
49     return 0;
50 }

```

Listing 8: 例题 3: 航班预订统计 - 差分数组

5.4 例题 4：拼车问题

题目描述：给定一个行程数组 $trips[i] = [numPassengers, from, to]$ ，表示在 $from$ 公里处上车 $numPassengers$ 人，在 to 公里处下车。判断是否能够一次接送所有乘客且不超载。

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      bool carPooling(vector<vector<int>>& trips, int capacity) {
6          // 找出最大的下车点
7          int maxPos = 0;
8          for (auto& trip : trips) {
9              maxPos = max(maxPos, trip[2]);
10         }
11
12         // 差分数组
13         vector<int> diff(maxPos + 2, 0);
14
15         // 处理每次行程
16         for (auto& trip : trips) {
17             int num = trip[0];
18             int from = trip[1];
19             int to = trip[2];
20
21             // 在from处上车
22             diff[from] += num;
23             // 在to处下车
24             diff[to] -= num;
25         }
26
27         // 模拟行驶过程，检查是否超载
28         int curPassengers = 0;
29         for (int i = 0; i <= maxPos; i++) {
30             curPassengers += diff[i];
31             if (curPassengers > capacity) {
32                 return false; // 超载
33             }
34         }
35     }
```

```

36         return true;
37     }
38
39     int main() {
40         vector<vector<int>> trips = {
41             {2, 1, 5},
42             {3, 3, 7}
43         };
44         int capacity = 4;
45
46         cout << "容量" << capacity << ": ";
47         if (carPooling(trips, capacity)) {
48             cout << "可以完成所有行程" << endl;
49         } else {
50             cout << "会超载" << endl;
51         }
52         // 输出: 会超载 (1-3公里有2人, 3-5公里有5人>4)
53
54         capacity = 5;
55         cout << "容量" << capacity << ": ";
56         if (carPooling(trips, capacity)) {
57             cout << "可以完成所有行程" << endl;
58         } else {
59             cout << "会超载" << endl;
60         }
61         // 输出: 可以完成所有行程
62
63         return 0;
64     }

```

Listing 9: 例题 4: 拼车问题 - 差分数组

6 进阶题目详解

6.1 进阶题 1: 乘积最大子数组

题目描述: 给定一个整数数组, 找出乘积最大的连续子数组, 返回其乘积。

解题思路: 这是前缀思想的扩展。需要同时维护前缀最大值和前缀最小值 (因为负数乘以负数可能变成最大正数)。

```

1      #include <iostream>
2      #include <vector>
3      #include <algorithm>
4      using namespace std;
5
6      int maxProduct(vector<int>& nums) {
7          if (nums.empty()) return 0;
8
9          int maxProd = nums[0]; // 全局最大值
10         int curMax = nums[0]; // 以前一个位置结尾的最大乘积
11         int curMin = nums[0]; // 以前一个位置结尾的最小乘积（处
            理负数）
12
13         for (int i = 1; i < nums.size(); i++) {
14             // 如果当前数是负数，交换最大和最小
15             if (nums[i] < 0) {
16                 swap(curMax, curMin);
17             }
18
19             // 更新当前位置的最大最小值
20             curMax = max(nums[i], curMax * nums[i]);
21             curMin = min(nums[i], curMin * nums[i]);
22
23             // 更新全局最大值
24             maxProd = max(maxProd, curMax);
25         }
26
27         return maxProd;
28     }
29
30     int main() {
31         vector<int> nums1 = {2, 3, -2, 4};
32         cout << "数组 [2,3,-2,4] 的最大乘积: "
33         << maxProduct(nums1) << endl; // 6
34
35         vector<int> nums2 = {-2, 0, -1};
36         cout << "数组 [-2,0,-1] 的最大乘积: "
37         << maxProduct(nums2) << endl; // 0
38
39         vector<int> nums3 = {-2, 3, -4};

```

```

40         cout << "数组 [-2,3,-4] 的最大乘积: "
41         << maxProduct(nums3) << endl;    // 24
42
43         return 0;
44     }

```

Listing 10: 进阶题 1: 乘积最大子数组

6.2 进阶题 2: 连续子数组的最大和

题目描述: 给定一个整数数组，找出具有最大和的连续子数组，返回其最大和。

```

1     #include <iostream>
2     #include <vector>
3     #include <algorithm>
4     using namespace std;
5
6     // 方法1: 动态规划 (Kadane 算法)
7     int maxSubArrayDP(vector<int>& nums) {
8         int curMax = nums[0];
9         int globalMax = nums[0];
10
11         for (int i = 1; i < nums.size(); i++) {
12             curMax = max(nums[i], curMax + nums[i]);
13             globalMax = max(globalMax, curMax);
14         }
15
16         return globalMax;
17     }
18
19     // 方法2: 前缀和优化
20     int maxSubArrayPrefix(vector<int>& nums) {
21         int minPrefixSum = 0;    // 最小前缀和
22         int prefixSum = 0;       // 当前前缀和
23         int maxSum = nums[0];    // 最大子数组和
24
25         for (int num : nums) {
26             prefixSum += num;
27             // 当前子数组和 = 当前前缀和 - 最小前缀和
28             maxSum = max(maxSum, prefixSum - minPrefixSum);
29             // 更新最小前缀和

```

```

30         minPrefixSum = min(minPrefixSum, prefixSum);
31     }
32
33     return maxSum;
34 }
35
36 int main() {
37     vector<int> nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
38
39     cout << "数组: ";
40     for (int x : nums) cout << x << " ";
41     cout << endl;
42
43     cout << "最大子数组和(DP): " << maxSubArrayDP(nums) <<
44         endl; // 6
45     cout << "最大子数组和(前缀和): " << maxSubArrayPrefix(
46         nums) << endl; // 6
47     // 子数组: [4, -1, 2, 1]
48
49     return 0;
50 }

```

Listing 11: 进阶题 2: 最大子数组和 - 前缀和优化

6.3 进阶题 3: 矩形区域不超过 K 的最大和

题目描述: 给定一个非空二维矩阵和一个整数 k , 找到矩阵中矩形区域的和不超过 k 的最大值。

解题思路:

- 固定上下边界, 使用一维前缀和 + 有序集合
- 将二维问题转化为一维问题
- 对于每个固定上下边界的子矩阵, 在一维前缀和中二分查找

```

1     #include <iostream>
2     #include <vector>
3     #include <set>
4     #include <algorithm>
5     #include <climits>

```

```

6      using namespace std;
7
8      int maxSumSubmatrix(vector<vector<int>>& matrix, int k) {
9          if (matrix.empty()) return 0;
10
11         int n = matrix.size();
12         int m = matrix[0].size();
13         int maxSum = INT_MIN;
14
15         // 固定左边界
16         for (int left = 0; left < m; left++) {
17             vector<int> rowSum(n, 0); // 每行的累计和
18
19             // 扩展右边界
20             for (int right = left; right < m; right++) {
21                 // 更新每行的累计和
22                 for (int i = 0; i < n; i++) {
23                     rowSum[i] += matrix[i][right];
24                 }
25
26                 // 在一维数组中找不超过k的最大子数组和
27                 set<int> prefixSet;
28                 prefixSet.insert(0);
29                 int curSum = 0;
30
31                 for (int sum : rowSum) {
32                     curSum += sum;
33
34                     // 找最小的前缀和 >= curSum - k
35                     auto it = prefixSet.lower_bound(curSum - k);
36                     if (it != prefixSet.end()) {
37                         maxSum = max(maxSum, curSum - *it);
38                     }
39
40                     prefixSet.insert(curSum);
41                 }
42             }
43         }
44
45         return maxSum;

```



```

46     }
47
48     int main() {
49         vector<vector<int>> matrix = {
50             {1, 0, 1},
51             {0, -2, 3}
52         };
53         int k = 2;
54
55         cout << "矩阵中不超过" << k << "的最大矩形和: "
56         << maxSumSubmatrix(matrix, k) << endl;
57         // 输出: 2
58
59         return 0;
60     }

```

Listing 12: 进阶题 3: 矩形区域不超过 K 的最大数值和

6.4 进阶题 4: 区间加法（二维差分应用）

题目描述：多次对二维数组的子区域进行加法操作，最后返回修改后的数组。

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     vector<vector<int>> rangeAddQueries(int n,
6     vector<vector<int>>& queries) {
7         // 二维差分数组（多开一行一列）
8         vector<vector<int>> diff(n + 1, vector<int>(n + 1, 0));
9
10        // 处理每个查询
11        for (auto& q : queries) {
12            int r1 = q[0], c1 = q[1];
13            int r2 = q[2], c2 = q[3];
14
15            diff[r1][c1] += 1;
16            diff[r2 + 1][c1] -= 1;
17            diff[r1][c2 + 1] -= 1;
18            diff[r2 + 1][c2 + 1] += 1;
19        }

```

```

20
21 // 还原矩阵
22 vector<vector<int>> result(n, vector<int>(n, 0));
23 for (int i = 0; i < n; i++) {
24     for (int j = 0; j < n; j++) {
25         result[i][j] = diff[i][j];
26         if (i > 0) result[i][j] += result[i-1][j];
27         if (j > 0) result[i][j] += result[i][j-1];
28         if (i > 0 && j > 0) result[i][j] -= result[i-1][j
        -1];
29     }
30 }
31
32 return result;
33 }
34
35 int main() {
36     int n = 3;
37     vector<vector<int>> queries = {
38         {0, 0, 1, 1},
39         {0, 0, 1, 0},
40         {1, 1, 2, 2}
41     };
42
43     auto result = rangeAddQueries(n, queries);
44
45     cout << "结果矩阵:" << endl;
46     for (auto& row : result) {
47         for (int val : row) {
48             cout << val << " ";
49         }
50         cout << endl;
51     }
52
53     return 0;
54 }

```

Listing 13: 进阶题 4: 区间加法 II - 二维差分

6.5 进阶题 5：按权重随机选择

题目描述：给定一个正整数数组 w ，其中 $w[i]$ 代表第 i 个下标的权重。实现一个函数随机地返回下标，使得下标 i 被选取的概率与其权重成正比。

```
1      #include <iostream>
2      #include <vector>
3      #include <random>
4      #include <algorithm>
5      using namespace std;
6
7      class Solution {
8      private:
9          vector<int> prefixSum; // 前缀和数组
10         int totalSum;
11
12     public:
13         Solution(vector<int>& w) {
14             int n = w.size();
15             prefixSum.resize(n);
16
17             prefixSum[0] = w[0];
18             for (int i = 1; i < n; i++) {
19                 prefixSum[i] = prefixSum[i-1] + w[i];
20             }
21
22             totalSum = prefixSum.back();
23         }
24
25         int pickIndex() {
26             // 生成1到totalSum之间的随机数
27             int target = rand() % totalSum + 1;
28
29             // 二分查找第一个 >= target 的前缀和位置
30             int left = 0, right = prefixSum.size() - 1;
31             while (left < right) {
32                 int mid = left + (right - left) / 2;
33                 if (prefixSum[mid] >= target) {
34                     right = mid;
35                 } else {
36                     left = mid + 1;
```

```

37         }
38     }
39
40     return left;
41 }
42 };
43
44 int main() {
45     vector<int> w = {1, 3, 2};
46     Solution solution(w);
47
48     // 模拟1000次选择
49     vector<int> count(3, 0);
50     for (int i = 0; i < 10000; i++) {
51         count[solution.pickIndex()]++;
52     }
53
54     cout << "权重: [1, 3, 2]" << endl;
55     cout << "模拟10000次的结果:" << endl;
56     for (int i = 0; i < 3; i++) {
57         cout << "索引" << i << ": " << count[i]
58             << "次 (" << (double)count[i]/100 << "%)" << endl;
59     }
60     // 索引1应该出现约50%，索引2约33%，索引0约17%
61
62     return 0;
63 }

```

Listing 14: 进阶题 5: 按权重随机选择 - 前缀和 + 二分

7 高级技巧与优化

7.1 技巧 1: 前缀和 + 单调栈

```

1     #include <iostream>
2     #include <vector>
3     #include <stack>
4     #include <algorithm>
5     #include <climits>

```

```

6      using namespace std;
7
8      // 和大于等于K的最短子数组（包含负数）
9      int shortestSubarray(vector<int>& nums, int k) {
10         int n = nums.size();
11
12         // 计算前缀和
13         vector<long long> pre(n + 1, 0);
14         for (int i = 0; i < n; i++) {
15             pre[i + 1] = pre[i] + nums[i];
16         }
17
18         // 单调双端队列（存储索引）
19         deque<int> dq;
20         int minLen = INT_MAX;
21
22         for (int i = 0; i <= n; i++) {
23             // 检查是否有满足条件的子数组
24             while (!dq.empty() && pre[i] - pre[dq.front()] >= k)
25             {
26                 minLen = min(minLen, i - dq.front());
27                 dq.pop_front();
28             }
29
30             // 维护前缀和单调递增
31             while (!dq.empty() && pre[i] <= pre[dq.back()]) {
32                 dq.pop_back();
33             }
34
35             dq.push_back(i);
36
37             return minLen == INT_MAX ? -1 : minLen;
38         }
39
40         int main() {
41             vector<int> nums = {2, -1, 2};
42             int k = 3;
43
44             cout << "数组: 2 -1 2" << endl;

```

```

45     cout << "和>=" << k << "的最短子数组长度: "
46     << shortestSubarray(nums, k) << endl;
47     // 输出: 3
48
49     return 0;
50 }

```

Listing 15: 前缀和 + 单调栈: 和大于 target 的最短子数组

7.2 技巧 2: 前缀和 + 同余定理

```

1     #include <iostream>
2     #include <vector>
3     #include <unordered_map>
4     using namespace std;
5
6     int subarraysDivByK(vector<int>& nums, int k) {
7         unordered_map<int, int> modCount;
8         modCount[0] = 1; // 前缀余数为0出现1次
9
10        int curSum = 0;
11        int count = 0;
12
13        for (int num : nums) {
14            curSum += num;
15
16            // 处理负数的情况
17            int mod = (curSum % k + k) % k;
18
19            // 同余定理: 如果两个前缀和的余数相同, 则它们之间的子
                数组和能被k整除
20            if (modCount.find(mod) != modCount.end()) {
21                count += modCount[mod];
22            }
23
24            modCount[mod]++;
25        }
26
27        return count;
28    }

```

```

29
30     int main() {
31         vector<int> nums = {4, 5, 0, -2, -3, 1};
32         int k = 5;
33
34         cout << "能被" << k << "整除的子数组个数: "
35         << subarraysDivByK(nums, k) << endl;
36         // 输出: 7
37
38         return 0;
39     }

```

Listing 16: 前缀和 + 同余：和可被 K 整除的子数组

7.3 技巧 3：前缀统计（计数排序思想）

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     vector<int> smallerNumbersThanCurrent(vector<int>& nums) {
6         // 计数数组（题目限制 0 <= nums[i] <= 100）
7         const int MAX_VAL = 101;
8         vector<int> count(MAX_VAL, 0);
9
10        // 统计每个数字出现的次数
11        for (int num : nums) {
12            count[num]++;
13        }
14
15        // 计算前缀和：prefix[i] 表示小于 i 的数字个数
16        vector<int> prefix(MAX_VAL, 0);
17        for (int i = 1; i < MAX_VAL; i++) {
18            prefix[i] = prefix[i-1] + count[i-1];
19        }
20
21        // 获取结果
22        vector<int> result(nums.size());
23        for (int i = 0; i < nums.size(); i++) {
24            result[i] = prefix[nums[i]];

```

```

25     }
26
27     return result;
28 }
29
30 int main() {
31     vector<int> nums = {8, 1, 2, 2, 3};
32
33     cout << "原始数组: ";
34     for (int x : nums) cout << x << " ";
35     cout << endl;
36
37     auto result = smallerNumbersThanCurrent(nums);
38
39     cout << "小于当前数字的个数: ";
40     for (int x : result) cout << x << " ";
41     cout << endl;
42     // 输出: 4 0 1 1 3
43
44     return 0;
45 }

```

Listing 17: 前缀统计：有多少小于当前数字的数字

8 总结与对比

前缀和与差分的适用场景总结

技术	操作	时间复杂度	适用场景
前缀和	区间查询	$O(1)$	多次查询、静态数组
差分	区间修改	$O(1)$	多次修改、最终查询
前缀和 + 哈希	统计特定和	$O(n)$	统计满足条件的子数组个数
二维前缀和	子矩阵查询	$O(1)$	二维矩阵多次查询
二维差分	子矩阵修改	$O(1)$	二维矩阵多次修改

解题思维框架

遇到以下特征时，考虑前缀和/差分：

前缀和标志：

- ”区间和”、”子数组和”
- ”连续子数组”
- ”查询区间内的元素”
- 需要 $O(1)$ 回答大量区间查询

差分标志：

- ”区间增减”、”区间修改”
- ”多次修改后查询”
- ”航班预订”、”拼车”这类问题
- 批量更新后获取最终结果

组合使用标志：

- ”前缀和 + 哈希表”：统计特定和的子数组
- ”前缀和 + 单调栈/队列”：限制条件下的最优子数组
- ”前缀和 + 二分”：按权重随机选择

常见陷阱与注意事项

1. 索引边界：前缀和数组通常需要多开一位，注意 $\text{pre}[0]=0$ 的处理
2. 溢出问题：前缀和可能超出 int 范围，使用 long long
3. 负数处理：哈希表中查找 $\text{curSum}-k$ 时，注意负数情况
4. 二维边界：二维前缀和/差分注意行列的对应关系
5. 差分还原：差分数组求前缀和才能还原，不要忘记最后一步

9 练习题推荐

1. 前缀和基础：

- 区域和检索-数组不可变 (LeetCode 303)
- 连续子数组的和 (LeetCode 523)
- 和为 K 的子数组 (LeetCode 560)
- 中心下标 (LeetCode 724)

2. 差分基础:

- 区间加法 (LeetCode 370)
- 航班预订统计 (LeetCode 1109)
- 拼车 (LeetCode 1094)
- 使数组变成目标数组的最少操作次数 (LeetCode 1526)

3. 二维应用:

- 二维区域和检索-矩阵不可变 (LeetCode 304)
- 矩阵区域和 (LeetCode 1314)
- 最大子矩阵 (LeetCode 面试题 17.24)

4. 进阶挑战:

- 矩形区域不超过 K 的最大数值和 (LeetCode 363)
- 和至少为 K 的最短子数组 (LeetCode 862)
- 按权重随机选择 (LeetCode 528)
- 连续数组 (LeetCode 525)